

Módulo 4: Objetos y modelos de datos

Contenidos clave

Clases, herencia, composición, dunder methods:

- Clase: una clase es un molde para crear objetos. Definen qué datos tienen y qué pueden hacer. Una clase agrupa datos y comportamientos.
- Herencia: la herencia permite crear una clase a partir de otra y reutilizar o modificar las características de la clase que hereda. “es un”.
- Composición: la composición ocurre cuando un objeto contiene otro objeto, en lugar de heredar, un objeto tiene otro objeto. “tiene un”.
- Dunder methods: son métodos especiales de Python cuyo nombre comienza y termina con __ (__init __, __str __, __eq __, __len __); permiten definir cómo se comportan los objetos frente a operaciones de Python.

dataclasses y attrs:

- dataclasses: sirve para crear clases principalmente destinadas a almacenar datos, evitando escribir mucho código repetitivo; genera automáticamente ciertos métodos como __init __, __repr __ y __eq __. Facilita crear modelos de datos sin escribir tanto código.
- attrs: es una librería externa que facilita crear clases orientadas a datos, ofrece más herramientas y opciones para construir clases complejas. Simplifica la creación de clases y permite configurar aspectos como validadores, convertidores y atributos.

Pydantic para validación/serialización: Pydantic sirve principalmente para validar y transformar datos utilizando anotaciones de tipo. Es especialmente útil cuando se reciben datos externos por ejemplo de un API. Valida datos de entrada y salida. Serialización es el proceso de convertir un objeto o estructura de datos de Python a un formato que pueda almacenarse o transmitirse, por ejemplo JSON.

Objetivos

- Modelar entidades con comportamientos y validaciones.

La entidad principal es:

```
38 @dataclass
39 class Order:
```

No solamente almacena datos, también tiene comportamiento.

```
45 @property
46 def total_amount(self) -> float:
47     return round(sum(item.subtotal for item in self.items), 2)
```

```
33 @property
34 def subtotal(self) -> float:
35     return round(self.price * self.quantity, 2)
```

Order calcula su propio total y OrderItem calcula su subtotal. Tienen comportamientos especiales mediante dunder methods:

```
51     def __eq__(self, other: object) -> bool:
52         if not isinstance(other, Order):
53             return NotImplemented
54         return self.id == other.id
55
56     def __lt__(self, other: Order) -> bool:
57         """Permite ordenar/comparar órdenes por su monto total."""
58         return self.total_amount < other.total_amount
```

Podemos realizar la comparación de las órdenes comparando sus montos totales.

Validación. Se establecen reglas y se valida que la orden tenga productos.

```
10     class ItemIn(BaseModel):
11         name: str = Field(..., min_length=1)
12         price: float = Field(..., gt=0)
13         quantity: int = Field(default=1, gt=0)
14
15     @field_validator("items")
16     @classmethod
17     def validar_items_no_vacios(cls, items: list[ItemIn]) -> list[ItemIn]:
18         if not items:
19             raise ValueError("La orden debe incluir al menos un producto")
20         return items
```

- Serializar y validar entradas/salidas.

El flujo es: Datos externos -> Order In -> Validación -> Entidad Order -> Lógica -> OrderOut -> Serialización -> JSON.

A la entrada se tiene un diccionario:

```
92     payload_raw = {
93         "customer_email": "cliente@empresa.com",
94         "items": [
95             {"name": "Monitor 274k", "price": 350.50, "quantity": 1},
96             {"name": "Cable HDMI", "price": 12.00, "quantity": 2},
97         ],
98     }
```

El cual se convierte en un modelo pydantic:

```
101     dto_in = OrderIn(**payload_raw)
```

Pydantic valida los datos y hace que el correo tenga formato válido.

Conversión a entidad. Convierte OrderIn en Order, y los ItemIn en OrderItem.

```
105     orden_1 = crear_orden_desde_dto("ORD-001", dto_in)
106
107     items_dominio = [
108         OrderItem(name=item.name, price=item.price, quantity=item.quantity)
109         for item in dto.items
110     ]
```

Se separan los modelos de entrada de la entidad de dominio. Después:

```
121     dto_out = exportar_orden_dto(orden_1)
```

Convierte Order en OrderOut y:

```
122     print(  
123         " JSON de Salida listo para API:", dto_out.model_dump_json(indent=2)  
124     )
```

Convierte el modelo Pydantic a JSON.

Laboratorio

- *dataclass Order con cálculos derivados y comparaciones.*

dataclass:

```
38     @dataclass  
39     class Order:
```

Tiene cálculos derivados:

```
45     @property  
46     def total_amount(self) -> float:  
47         return round(sum(item.subtotal for item in self.items), 2)
```

total_amount es un dato derivado, porque no se almacena directamente, se calcula a partir de los productos.

```
33     @property  
34     def subtotal(self) -> float:  
35         return round(self.price * self.quantity, 2)
```

subtotal hace lo mismo a nivel de OrderItem.

Comparaciones:

```
51     def __eq__(self, other: object) -> bool:
```

Compara igualdad con orden_1 == orden_2.

```
56     def __lt__(self, other: Order) -> bool:
```

Compara cuál tiene menor monto, orden_2 < orden_1.

La entidad de dominio denominada Order utiliza dataclass, con el objetivo de representar una orden de compra y encapsular tanto sus datos como comportamientos relacionados; contiene información como el identificador de la orden, correo del cliente, productos y fecha de creación. La clase OrderItem representa cada producto incluido en la orden.

subtotal calcula el importe de cada producto multiplicando su precio por la cantidad , y total_amount, que obtiene el importe total de la orden a partir de los subtotales de sus productos.

Los métodos especiales (Dunder methods) permiten la comparación entre órdenes. el método `__eq__` determina si dos órdenes corresponden al mismo identificador, mientras que `__lt__` permite comparar órdenes según su monto total.

- Modelos Pydantic (OrderIn/OrderOut) y conversión a entidad.

OrderIn/OrderOut:

```
16 class OrderIn(BaseModel): 62 class OrderOut(BaseModel):
```

Mapeadores:

```
71 def crear_orden_desde_dto(order_id: str, dto: OrderIn) -> Order:
81 def exportar_orden_dto(order: Order) -> OrderOut:
```

Los modelos Pydantic controlan los datos que ingresan y salen del sistema mediante OrderIn y OrderOut.

El modelo OrderIn representa la información recibida como entrada y permite aplicar validaciones automáticas sobre los datos, ejemplo de ello es la validación del correo electrónico, que tenga un formato válido, que los precios y cantidades sean mayores a cero y que la orden contenga al menos un producto.

Después con la función `crear_orden_desde_dto`, los datos validados del DTO de entrada se convierten en una entidad `Order`, separando los modelos utilizados para transportar información de la entidad usada dentro de la lógica de dominio.

Por último con `exportar_orden_dto`, la entidad `Order` se transforma nuevamente en un modelo `OrderOut`. Este modelo puede ser serializado a formato JSON con `model_dump_json()`, permitiendo preparar la información para ser enviada como respuesta de una API.

De esta forma se establece un flujo de validación, transformación y serialización de los datos:

Datos de entrada -> OrderIn -> Validación -> Order -> Lógica de dominio -> OrderOut -> JSON.